Specyfikacja Architektoniczna
Eden Node Exporter

Dokument Projektowy - System Monitoringu HPC

19 października 2026

Wstęp

Aplikacja została zaprojektowana w języku Go

1 Zarządzanie Cyklem Życia (Daemon i Singleton)

- **Zarządzanie przez systemd:** Program jest kompilowany do pojedynczego pliku binarnego i uruchamiany w tle za pomocą pliku usługi `.service`. Zapewnia to automatyczny start programu wraz z systemem operacyjnym oraz politykę `Restart=always` w przypadku nieoczekiwanego błędu.
- **Blokada instancji (Singleton):** Aby zapobiec uruchomieniu wielu instancji agenta fałszujących statystyki na jednym węźle, program wykorzystuje blokadę plikową na poziomie jądra systemu (`syscall.Flock`). Zastosowanie blokady jądra (w przeciwieństwie do czystego `O_EXCL`) zapobiega konfliktom – w przypadku nagłego zatrzymania zasilania, Linux samoczynnie zwalnia blokadę przypisaną do `/var/run/eden-agent.lock`.
- **Graceful Shutdown:** Program nasłuchuje na sygnały systemowe `SIGINT` oraz `SIGTERM`. Wymusza to bezpieczne zamknięcie połączenia ze sterownikami (NVML), wyczyszczenie pamięci oraz zwolnienie blokady przed fizycznym zakończeniem procesu.

2 Monitorowanie GPU (Biblioteka go-nvml)

- **Odporność na brak sprzętu:** Węzły mogą różnić się konfiguracją (niektóre posiadają wyłącznie procesory graficzne, inne są typu CPU-only). W przypadku braku układów graficznych, wywołanie `nvml.Init()` zwraca błąd, który jest przechwytywany. Program wchodzi wtedy łagodnie w tryb *CPU-only* i kontynuuje działanie.
- **Leniwa Inicjalizacja (Device Handles):** Uchwyty do kart graficznych są pobierane tylko raz na początku cyklu życia programu, aby uniknąć zbędnego obciążania API sterownika NVIDII przy każdym cyklu pomiarowym.
- **Zbierane metryki:** Z wykorzystaniem uchwytów pobierane są informacje o zajętości rdzeni obliczeniowych (%), zajętości VRAM oraz powiązanych numerach PID procesów działających na danym akceleratorze (funkcja `nvml.DeviceGetComputeRunningProcesses()`).

3 Zbieranie Danych CPU i RAM

- **Brak zależności zewnętrznych:** Program nie polega na zewnętrznych narzędziach (jak np. binarne wywołania oryginalnego skryptu *pestat*). Całość odczytów realizowana jest wewnętrznie poprzez bezpośredni odczyt plików.

- **Odczyt Cgroups:** Skrypt korzysta z natywnych mechanizmów kontroli zasobów Linuxa. Odczytuje statystyki z plików takich jak `cpuacct.usage` oraz `memory.usage_in_bytes` (lub ich odpowiedników dla Cgroups v2).
- **Obliczanie obciążenia:** Obciążenie procesora jest kalkulowane poprzez zachowanie poprzedniej wartości całkowitego czasu CPU, odczekanie interwału pomiarowego (np. 15 sekund) i podzielenie różnicy tych wartości przez upłynięty czas.

4 Odkrywanie Zadań i Cache Metadanych

- **Skanowanie Systemu Plików:** Agent listuje zawartość katalogu `/sys/fs/cgroup/system.slice/` celem znalezienia nowych katalogów należących do zadań Slurma. Z nazwy katalogu wyodrębniany jest identyfikator zadania (Job ID).
- **Mapowanie PID:** Wewnątrz katalogu zadania odczytywany jest plik `cgroup.procs`, zawierający pełną listę aktywnych procesów danego zadania. Pozwala to dopasować konkretne procesy systemowe do obciążenia na kartach GPU.
- **Mechanizm Buforowania (Lazy Caching):** Program utrzymuje w pamięci mapę łączącą identyfikatory na metadane (Użytkownik, Partycja, Nazwa). O serwer Slurma pyta przy pomocy komendy `scontrol show job` tylko i wyłącznie w momencie wykrycia nieznanego wcześniej Job ID (zjawisko Cache Miss).
- **Eksmisja Danych:** Jeśli w kolejnym cyklu pomiarowym folder przypisany do Job ID zniknie (zadanie zostało zakończone przez Slurma), identyfikator ten jest bezpiecznie usuwany ze słownika.

5 Optymalizacja Alokacji Pamięci (Zero-Allocation)

W celu zapobiegania nadmiernej pracy mechanizmu odśmiecania pamięci (Garbage Collector) w środowisku uruchomieniowym języka Go, program minimalizuje tworzenie nowych obiektów w czasie trwania pętli monitorującej.

- Zamiast odpytywać system z użyciem funkcji alokujących pamięć dynamicznie, agent jednorazowo rezerwuje stałą tablicę (bufor pamięci o rozmiarze np. 4096 bajtów) przed rozpoczęciem głównej pętli.
- Kolejne odczyty z wirtualnego systemu plików ładują zawartość strumienia tekstowego bezpośrednio do wyżej wspomnianego, pre-alokowanego bufora, który jest cyklicznie nadpisywany. Minimalizuje to wahania w zużyciu pamięci RAM przez samego demona.

6 Eksport Danych (Model Pull / Prometheus)

Proces raportowania został zbudowany w architekturze eksportu pasywnego.

- Agent nie otwiera aktywnych połączeń i nie wysyła danych w sposób ciągły do zewnętrznej bazy. Zamiast tego utrzymuje bezstanowy serwer HTTP działający w tle (np. nasłuchujący na porcie 9100).
- Używa standardowej biblioteki `client_golang/prometheus`. W momencie zeskrapowania portu przez główny serwer metryk klastra, agent błyskawicznie zwraca w czasie rzeczywistym sformatowane, gotowe wartości opatrzone precyzyjnymi tagami (nazwa węzła, Job ID, nazwa użytkownika, indeks GPU). Odciąża to węzły obliczeniowe i gwarantuje stabilność raportowania.